

Coding Interview Mastery

To make a copy of this document click [here](#), then go to File → Make a Copy.

Golden Idea

Coding interviews are essential to landing a software engineering internship or full-time job offer. This video is going to teach you everything you need to know to master them.

Importance

What's the main reason this idea is important to understand?

- Nearly every single software engineering interview process will have at least one coding interview — sometimes, they'll have 3-6 of them depending on how competitive & prestigious the company is.
- This is why we need you to have the capability to handle them & hold your own in a live interview setting.

Table of Contents

[What is a Coding Interview?](#)

[Structure](#)

[What Kinds of Problems?](#)

[Full Structure & Timeline of a Problem](#)

[Techniques](#)

[Implement](#)

[Review](#)

[Conclusion](#)

(See outline for additional information)



What is a Coding Interview?

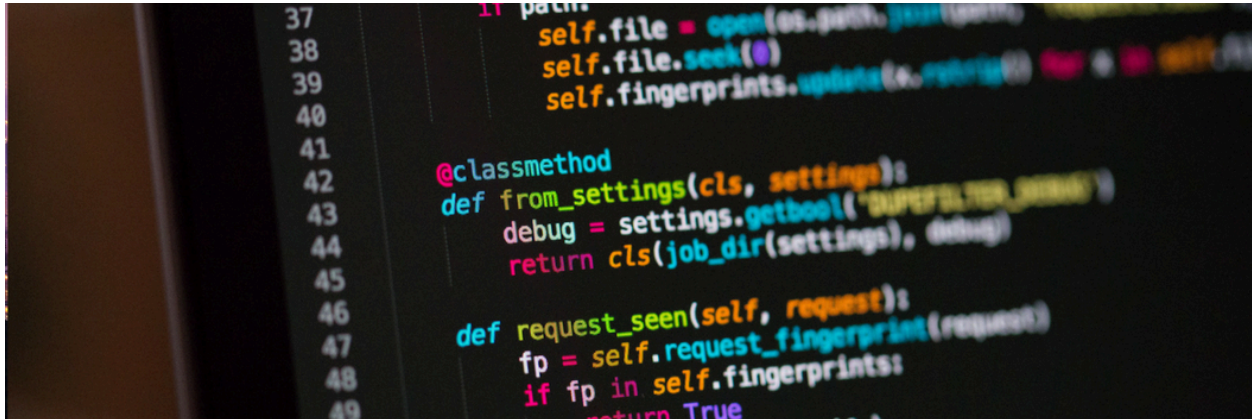
A **coding interview** is a technical assessment used by companies, primarily in the tech industry, to evaluate a candidate's problem-solving and coding skills. Typically, these interviews focus on data structures and algorithms (DSA) problems, most commonly LeetCode medium level problems.

- **High-Level Structure:**
 - Most coding interviews last around 45-50 minutes and involve solving 1-2 problems, with about 25-30 minutes allocated per question. The structure can vary slightly, as some interviews may start with behavioral questions or reserve time at the end for additional discussion. The goal is to assess not only your coding ability but also how you approach problems, communicate your thought process, and optimize your solutions under time constraints.

The live coding interview is also usually done with 1-2 software engineers who will assess your performance and decide whether you move forward.

Structure

- (Optional: start with behavioral questions)
- Show you the problem (DS&A, OOP, or System Design)
- You'll have about 25-30 minutes to walk through and solve it live in front of them on a shared coding pad or whiteboard.
 - Note: shared coding pad (like Google Doc for coding) is the most common — you will not be able to handwrite anything, everything will be done on the shared coding pad.



What Kinds of Problems?

There are three options for these kinds of problems.

1. Data Structures & Algorithms Problem (Most Popular)

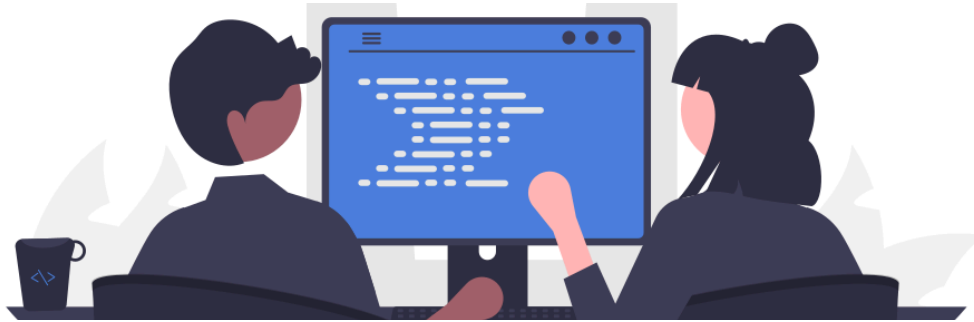
- Most of the time, you'll get a classic DS&A problem. This will most likely be at the level of a **LeetCode medium** (easier or harder depending on how competitive the role/company is).
- Designed to test your ability to solve algorithmic challenges using efficient data structures and problem-solving techniques. These problems typically assess:
 - Problem-Solving Skills:** How well you break down complex problems into smaller, manageable steps and apply logical reasoning to find solutions.
 - Knowledge of Data Structures:** Your understanding of common data structures like arrays, linked lists, stacks, queues, hash maps, trees, graphs, heaps, and how to use them effectively.
 - Algorithmic Thinking:** Your ability to apply algorithms such as sorting, searching, dynamic programming, backtracking, recursion, and greedy algorithms to solve problems efficiently.
 - Time and Space Complexity:** How well you can optimize solutions for speed (time complexity) and memory usage (space complexity), often using Big O notation to analyze performance.
 - Edge Cases and Testing:** Your ability to think about and handle edge cases, test the solution with various inputs, and debug effectively.
- [Pareto Problem Set](#) is what you can expect here (and NeetCode 150).

2. Object-Oriented Programming (OOP) Problems

- Test your understanding of designing systems using objects and classes, focusing on the core principles of OOP. These problems typically assess:
 1. **Understanding of OOP Concepts:** How well you grasp fundamental OOP principles like **encapsulation**, **inheritance**, **polymorphism**, and **abstraction**, and can apply them in practical coding scenarios.
 2. **System Design and Architecture:** Your ability to design scalable and maintainable software by defining relationships between classes, designing hierarchies, and organizing code into reusable components.
 3. **Class Design:** Creating classes with appropriate attributes and methods, implementing constructors, destructors, and handling object interaction.
 4. **Code Reusability and Extensibility:** How well you design systems that can be easily extended or modified in the future without breaking existing functionality.
- In OOP-focused interviews, problems often involve designing classes or systems like designing a library system, parking lot, LRU cache, or banking application, and require a balance of clean design and implementation.
- Note: You'll sometimes have a combination of DSA & OOP — “Implement a class that does X, which would need certain data structures & algorithms to do efficiently.”

3. System Design

- These are a lot less likely to show up at this stage which is why I will only briefly touch them here — they are more common for high level roles, but you should still have them on your radar. We will deep dive in a later video.



Full Structure & Timeline of a Problem

Here's a structured timeline for handling a coding problem in a 25-minute coding interview. These timings are an estimate and might change based on how long they give you (more or less time). Additionally, sometimes the sections bleed into each other.

0-5 Minutes: Clarifying and Understanding the Problem

- **Read the problem thoroughly:** Make sure you understand what's being asked.
- **Ask clarifying questions:** Clarify input/output formats, edge cases, and constraints.
- **Restate the problem:** Confirm your understanding with the interviewer by restating the problem in your own words.

5-12 Minutes: Solution Brainstorming and Planning

- **Discuss potential solutions:** Walk through different approaches, considering time and space complexity.
- **Compare and select the best solution:** Identify the most efficient solution for the given problem and edge cases.
- **Write out the plan:** Outline the steps or pseudocode before jumping into implementation.

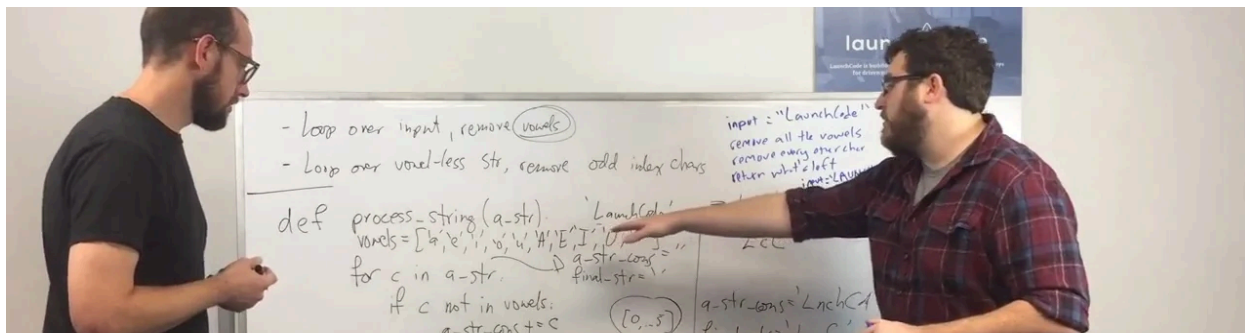
12-20 Minutes: Code Implementation

- **Start coding:** Write clean, efficient code based on your solution.
- **Explain as you code:** Keep explaining your thought process and logic to the interviewer.
- **Test edge cases:** While coding, think of edge cases and ensure your solution covers them.

20-25 Minutes: Review and Optimize

- **Review your solution:** Walk through the code with the interviewer to ensure it's correct.
- **Discuss improvements:** Suggest any optimizations or improvements if time permits.

- **Answer questions:** Be prepared to answer follow-up questions from the interviewer on the solution or alternative approaches.



💡 Techniques

Let's now go through each section & give you all the techniques and tips to have the best performance possible. We'll use Two Sum as an example for all of this.

High Level Point

- Take notes at EACH section using Commenting in the code. The interviewer can't see your handwriting, so you need to write down all your thoughts in bullet points in a comment as you discuss.

Understand

- **The purpose of this section is to make sure you fully understand the problem in all aspects BEFORE discussing any kind of solution.**
 - A very common mistake people make is they write a solution to the wrong problem — this happens ALL the time, and is something you really really want to avoid. It's a death sentence.
 - Make sure you understand the problem 100%, by asking clarifying questions and reading closely.

You MUST read the problem out loud. Do not silently read it in your head — get used to reading out loud & pausing for questions.

- You'll read a few lines, pause, rephrase, ask a clarifying question or two if you don't understand and continue.
 - I'm not saying spend tons of time on this — but do not move forward until you really understand all aspects of the problem.
- **Use the provided example & come up with at least one additional example to make sure you fully understand.**



- Usually, they will give you an example for input & output so you understand how the problem is set up.
- You need to make sure you fully get how this example works — and then also quickly write up another example and make sure you understand that.
- Write the new example in a new comment.

Consider edge cases

- An **edge case** is a scenario that occurs at the extreme limits or boundaries of the possible inputs for a problem. It tests the behavior of an algorithm under unusual or uncommon conditions, such as very large or small inputs, empty inputs, or inputs with unexpected formats.
 - Your code **NEEDS** to handle these properly, so you must ask about them early on.
- You'll want to think of edge cases & ask how they would work when clarifying the bounds of the problem.
 - Often, they'll just say something like "you can assume that the input follows the correct format," but by asking you set yourself apart as someone who isn't impulsive and really takes their time to understand the problem.
- Make sure to write ALL of these down in your Comment — as in, when the interviewer answers your edge case question, note that down quickly in the Comment.

Example Edge Cases

Not all of these will be relevant to the problem, but I want you to understand all options. Just run through the list & make sure you know what to do in every case.

1. Empty Input:

- What happens if the input is empty (e.g., an empty array, string, or null input)?

2. Single Element:

- How does the solution handle an input with just one element?

3. Minimum and Maximum Values:

- What happens with very small or very large input values (e.g., integer overflow, negative numbers)?

4. Duplicates:

- How does the solution deal with repeated elements or duplicates in the input?

5. Sorted Input:

- What happens if the input is already sorted, either in ascending or descending order?

6. Reverse Input:

- How does the solution perform if the input is in the opposite or reverse order of what's expected?

7. All Elements the Same:

- What happens if all elements in the input are the same (e.g., all values in an array are equal)?

8. Invalid Input:

- What if the input contains unexpected values or types (e.g., negative indices, null values, or out-of-bounds access)?

9. Large Input Size:

- How does the solution perform with extremely large inputs, testing for efficiency and scalability?

10. Boundary Conditions:

- Does the solution handle boundary conditions correctly (e.g., the smallest and largest possible inputs)?

11. Edge Conditions in Graphs/Trees:

- How does the solution handle cases with no nodes, a single node, or unconnected graphs?

12. Zeroes and Negative Numbers:

- What happens with zero values or negative numbers in calculations or comparisons?

13. Memory Limits:

- Does the solution efficiently handle memory, especially for large inputs or recursive solutions that could hit stack limits?

14. Cyclic Data:

- Does the algorithm correctly detect or handle cycles (e.g., in linked lists, graphs)?

15. Concurrency or Race Conditions:

- For multi-threaded problems, how does the solution handle simultaneous access or race conditions?

- These are standard edge cases — very often, you'll have **problem specific edge cases** so you'll need to keep asking clarifying questions.
 - ex. If the problem is about checking books out from a library, a clarifying question could be *"are there multiple copies of the same book?"*
- Pay attention to time — you want to make sure you fully understand the problem, but ensure that you're **not spending more than 5 mins on this part.**

Ideate

This is where you come up with viable solutions for the problem and discuss the pros/cons of each.

Note: you only proceed to this step once it's clear that you fully understand the problem.

- The first step of proper ideation is presenting several possible solutions & evaluating their efficiency.
 - Usually, I recommend producing a Brute Force solution to begin with and then evaluating its time & space complexity.
 - A **brute force solution** in a coding interview refers to the most straightforward or basic approach to solving a problem, typically by checking all possible solutions without any optimization. This method often involves iterating through all possible combinations or calculations to arrive at an answer, regardless of efficiency.
 - **We start with brute force for two reasons:**
 - It helps you think of a more optimal solution if you don't see it already.
 - It gives you a working solution to run with if you can't think of the correct one.
- We use time & space complexity to evaluate how good a potential solution is.
 - **Time complexity** refers to how the amount of time an algorithm takes to run increases as the size of the input grows. In simple terms, it measures how efficiently an algorithm performs as the input gets larger. For example, if an algorithm takes twice as long to run when the input doubles, its time complexity might be described as $O(n)$, where n is the size of the input.
 - **Space complexity** refers to how much memory or space an algorithm uses as the input size grows. It looks at the amount of extra memory needed beyond the input data. For instance, an algorithm that needs extra memory proportional to the size of the input has a space complexity of $O(n)$.
 - These concepts are VERY important — if you don't understand them fully, you should watch this video & really deep dive into them.

[Big-O Notation - For Coding Interviews](#)

- **The biggest mistake people make in coding interviews is not planning their solution in enough detail.**
 - Basically, people will pull out random data structures & ideas without planning to the end — making sure it actually works.
 - They'll then proceed to jump right into implementation, code a wrong solution, and then proudly announce, "I'm done!"
 - That's probably the worst thing you can possibly do in a coding interview.
 - It's difficult to describe what this is, but try your goddamn best to break your idea of a solution BEFORE coding anything at all. You need to make sure your code is rock solid before implementation.
- **Take detailed notes on these potential solutions.**
 - Write out at a high level how it would work, then write a series of bullet points explaining all the nuance, and end with a time/space complexity note so you can compare the solutions.
 - Write out 2-3 additional test cases and ensure that your solution would actually work for all of them.
 - You don't need to write full pseudocode, but I want to see that you've fully thought through all aspects of your plan in a 4-5 lines of explanation.
 - Again — the biggest mistake people make is implementing something that is clear doesn't actually work.
- **Continue speaking out loud, and don't go silent for more than 5-10s max.**
 - This is something you're probably not used to doing, so you should start practicing this when doing your routine LeetCode problems.
 - *Example: what does speaking aloud while thinking sound like?*
 - Note: Don't say tons of filler words like "hmm, umm, hmm," — that's not what I'm saying. I'm saying take your mental line of thought and just speak it aloud.
 - Don't say "I need a second to think."

- **Avoid shopping around for Data Structures**

- Another big mistake people make is just throwing random Data Structures at the problem thinking that they'll find a magic bullet without having to think about it
- "A LinkedList! no, a hash table! hmm, an array? a heap?" — this shows that you don't actually understand the problem, solution, or data structures — that kind of thinking will really impact your chances of passing.
- Buzzwords don't work — you need to actually understand WHY the data structure you pick to utilize would actually work. In addition, the best solutions often involve multiple data structures working together to be as efficient as possible, so you can't just randomly pick one and try it.

- **Avoid overfitting to the given example**

- Another mistake people make is they'll modify their solution to just fit the ONE provided example.
- *example: If the example starts with [0] as the first element in a list of the input, and that breaks their code for some reason, they'll just add a conditional check edge case just for the 0 and move on.*
 - This is wrong because you're not considering WHY a 0 breaks your code, and just putting a bandaid on it rather than fixing the actual issue.
 - Chances are, other test cases will break your code and you won't even realize.

- **Conditional Overload**

- People add hardcoded conditionals left and right to handle test case after test case without realizing that a clean solution usually works WITHOUT bonus conditionals.



- **Pick a solution and explain why.**

- At this point, you should have listed down 2-3 potential solutions *that work for the problem*. Now, you decide which one you're going to go with & explain WHY you made that choice.
 - Ideally, use the time & space complexity metrics to justify your choice.
 - Finally, keyword "that work" — make sure your solution actually works for the edge cases, tests, and examples before picking it.
- Once you've chosen a solution, say — "It seems like [solution name] is the best choice for now based on [reasons]. Is it okay if I implement it now?"

- **When None Of Your Solutions Work**

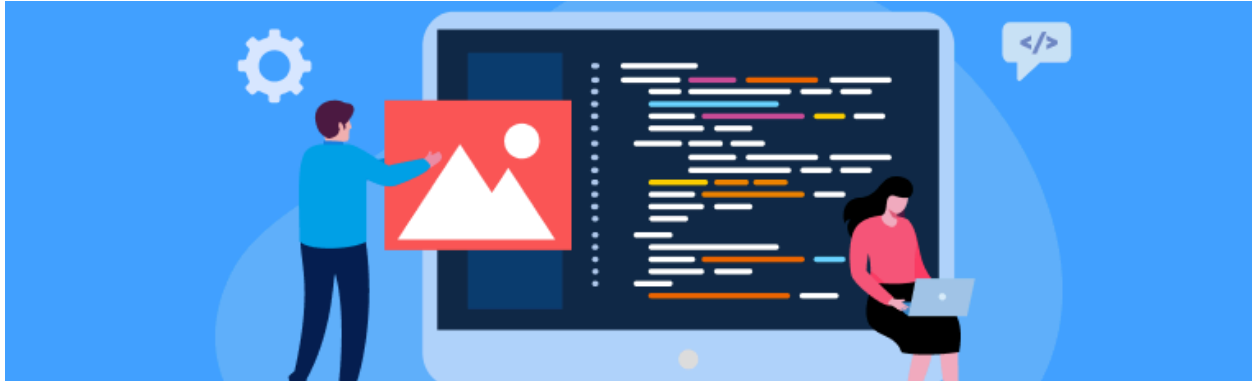
- At this point, you may face a problem where you cannot come up with a solution that reliably works for all test cases.
 - One possibility is that you think your solution works but doesn't — the whole point of this Ideate stage is to not let this happen (through examples, edge cases, planning to the end, etc.)
 - However, sometimes you can only think of 2-3 methods & can unfortunately come up with cases that break each of them.
- If you're earlier than minute 12 or 13, then keep ideating and trying to think of a solution that does work. But if you can't and time is running out, here's what to do
 - You must write out your solutions AND the cases that break it and the ones that it works for, for each solution.
 - Then, you'll say something like this:
 - *"At this point, here are several solutions that I can think of that can address the problem — unfortunately, while each of them work for some cases, I also have different test cases that don't pass for each of them. In the spirit of time, I wanted to ask — should I keep trying to think of one that works for all cases, or should I just pick one and roll with it so that we at least have a solution that works for some of the cases?"*
 - *"Got it — I'm leaning towards solution Y for X reason, so I'll go ahead and start coding it up. Thanks for the suggestion."*
 - It's important to understand that **even if none of your solutions are perfect**, you need to pick something and implement it. We really don't want you to end a coding interview without having a solution written that works for at least SOME of the cases.
 - Just do your best here.

- That's what you'll want to say. You also have an opportunity here to ask for a hint or advice.
 - Note: do not ask for a hint or direction before this point. You want to fully explore multiple paths as much as you possibly can BEFORE asking for any help.
 - *"Are any of these somewhat on the right track? Or would you recommend I scrap them all and go for another option entirely?"*
 - That is a good way to subtly ask for a hint — again, don't say "I need a hint" and don't ask too early.

- **Damage Control**

- "Action taken to limit the damaging* effects of an accident or error."
- When you're at the point that you don't have a solution that works & you need to ask for a hint, **we're doing damage control** — no matter how subtle you are, you kind of already in a pretty bad spot, and may very well not pass.
- However, not all is lost — I have passed coding interviews in the past that I didn't get the right solution or needed a hint.
- If you need a hint or a suggestion & then immediately get the right answer and everything else is perfect, good chances you'll move forward. But if you get a hint and still don't get the right solution, you're pretty much screwed.
- Basically, we want to get the right answer without a hint every time — but that is just what to do if you absolutely need one.

*To inflict physical harm on (something) so as to impair its value, usefulness, or normal function.(Google)



Implement

At this point, you've decided which of your 2-4 solutions you're going to go ahead and actually code up.

- Just to verify, you MUST fully understand how it works and plan to the end — what that means is that you should have discussed how each and every step works in your solution, not just general (use this data structure) and move on. I don't want you to code something you don't understand.
- You should have also justified WHY you're picking that solution to the interviewer.

Reference Your Notes

If you've written out your notes for your detailed solution in a comment properly, you can now just execute on those ideas.

You shouldn't be doing much thinking — you're just doing what is written as you write out your code. It's kind of like how a machine acts after it compiles the code — now it's just a set of instructions to follow.

No “should I do this, do that, etc.” — your written solution should already cover all thinking.

- **Note:** you might realize new information as you're going, which is normal.
 - You might realize that you did something wrong, made a mistake, have a test case that doesn't work, etc.
 - When that happens, definitely adapt and start thinking — but the goal is to just execute.



Pick A Language

When you do coding interviews, majority of the time you'll have a language of choice that you're most comfortable with.

Your default language is defined as the language you practice in & use for the majority of coding interviews.

- There are four major options:
 - Python
 - Java
 - C++
 - JavaScript

You don't want to go with anything else other than those as your default language, UNLESS the role calls for a specific language (C, Golang, etc.). But in general, you want to have a language that you do everything in.

Python is BY FAR the best — so if you're deciding which language, go with Python. Just do all your LeetCode and prep in Python and force yourself to learn it. You NEED to learn it.

Here's a video that will give you the basics: [Python for Coding Interviews - Everything you need to Know](#)

However, if you're strapped for time and have an interview coming up soon, then just do the language you're most comfortable with.

When You've Made A Mistake / Realize Something

There's a good chance that, when you're implementing your solution, you might realize that you left something out, forgot a case, or didn't consider a certain aspect of the problem. At this point, you have two options:

1. You can set it aside in a comment, finish the solution, and then go back and address it.
 - You should pick this option when it's a small change — something that can be addressed at the end & doesn't fundamentally transform other aspects of your code.
 - This option also can be good if you're very short on time and you need to just get something down.
2. You could address it now and update your solution.
 - You should do this if 1) the realization impacts the way you were planning on writing the rest of your code in a large way or 2) you have a lot of extra time.



Looking Up Syntax & Documentation for your Language

Sometimes, you'll forget a method, function, data structure, some aspect of syntax, etc, as you're implementing your solution.

In this case, try to think for a second, but if you don't remember, be clear with your interviewer:

- ex. *"I'm trying to remember the exact Python function for sorting, but I'm not exactly sure what it is. Do you know what it is? Or is it okay if I look it up on the Python documentation website?"*
 - Sometimes, they'll give it to you, or they'll let you look it up.
 - You don't want to have too many of these come up, but 1-2 can be okay.

Resources that are allowed to access.

- **Language Official Documentation**
 - This is by far the best option. If you just use this, you're pretty much fine, and most interviewers are okay if you access this.
 - [Python Docs, Java Docs, etc.]
- **Stack Overflow**
 - This can sometimes be okay but is a little discouraged. It's 50-50 — sometimes they'll give you general information, but sometimes they also give you something you can copy paste in which is not the best.
 - I would stick to the official documentation and only go here if necessary.

What you CANNOT access

- **ChatGPT or any AI Tools**
 - These are completely not allowed in any circumstance, unless they explicitly tell you it's okay. Which is unlikely.
- **Google/Search Engines**
 - You cannot Google the problem or data structures to find the time/space complexity or understand how they work.
 - Some people will Google time complexity of functions, and that's not allowed.

In simple terms, you're only allowed to look at syntax specific issues and nothing else.



Write Clean Code

In general, when implementing, you want your code to be as clean as possible.

- This is another aspect they're looking for — someone with great attention to detail is always a better software engineer and is easier to work with.
- Clean code is code that is easy to read, understand, and maintain. It follows a consistent style, uses meaningful names, and is well-organized, making it straightforward for other developers to work with and modify.
- Clean code avoids redundancy, keeps functions and classes focused on a single task, and includes comments only where necessary to clarify complex logic.
 - The goal of clean code is to reduce confusion and errors, allowing for efficient collaboration and future improvements.

Clean Code Principles

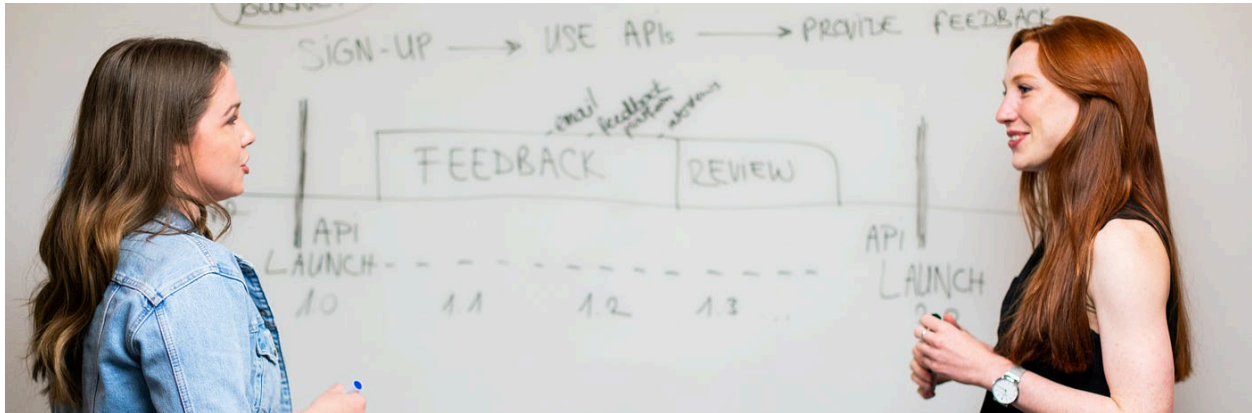
- **Meaningful Naming**: Use clear and descriptive variable, function, and class names that reflect their purpose. Avoid generic names like `temp` or `var`—instead, use names that convey intent.
- **Single Responsibility Principle**: Ensure that each function or method does only one thing and does it well. This makes your code modular, reusable, and easier to understand.
- **Keep it Simple**: Avoid complex logic, nested loops, or convoluted conditionals. Use straightforward, easy-to-follow code that solves the problem without unnecessary complications.
- **Consistent Formatting**: Maintain consistent indentation, spacing, and bracket placement to make the code visually easy to read and understand. Proper formatting helps the interviewer quickly follow your logic.
- **Avoid Hard-Coding**: Use parameters or constants instead of hard-coding values, and consider edge cases. This makes your code more flexible and maintainable, showing you can adapt to different scenarios.

Give Good Explanations As You Implement

Again, you really want to be explaining your code well as you write it. Make sure to write little comments here and there labeling what certain functions do, and it's better to over explain than under explain. Treat the interviewer like they're not very experienced and repeat yourself several times.

- “Just to summarize, this is what this part does..... any questions or clarifications about that?”
- As you implement a later section, you can revisit and explain how it connects back to what you did before.

Very often, interviewers will not be fully paying attention & even could be coding on the side. You want to make their life as easy as possible.



Review

At this point, you should have successfully implemented your chosen solution in full. Here are your next steps at this point to finish a problem strong.

Run It

- Sometimes, you'll actually be able to run your code (as there will be a bash/terminal window where you can call your function).
- If you can run it, run it! You'll encounter some syntax errors to debug, and then you'll even be able to run your test cases & print the return to see if you get the right answer.
- Chances are, you'll have some bugs in the logic that you'll need to fix through this process.

Walk Through Tests

- If you can't run your code, then pick 3 test cases where you'll manually walk through your solution and see if everything goes as you expect.
- Try to make your test cases diverse, and you can even come up with 1-2 new ones to be extremely sure your solution works.
- Fix your code if you see any issues.
- Also, do this all in comments — so write down your tests, step through, keep track of variable values, etc.

Discuss Optimizations

- At this point, your solution should work & you've tested it.
- Now, you'll want to talk through time and space complexity and offer up suggestions for how you would improve your code if you had more time.



- Here are some general suggestions to think about ways to improve your code. You want to avoid saying “I can’t think of anything to improve!”
 - **Time Complexity**: Review your algorithm's time complexity. Aim to reduce it if possible by eliminating unnecessary loops, minimizing nested operations, or using more efficient data structures.
 - **Space Complexity**: Evaluate your code’s space usage. Look for ways to use less memory, such as avoiding extra data structures or reusing variables when possible.
 - **Edge Cases and Inputs**: Consider how the code performs with edge cases, large inputs, and boundary values. Optimize your solution to handle these scenarios efficiently without causing performance issues.
 - **Remove Redundancy**: Identify and eliminate redundant calculations or repeated work. Cache values or use memoization to prevent recalculating the same results.
 - **Optimize Data Structures**: Check if the data structures you’re using are the most efficient for the task. For example, replacing lists with sets for faster lookup, or using heaps for priority ordering.
 - **Algorithmic Improvements**: Look for alternative algorithms that may offer better performance. For example, changing a brute-force approach to a divide-and-conquer strategy could significantly improve efficiency.
 - **In-Place Modifications**: If applicable, modify data in place rather than creating new copies to save memory. This is especially useful in problems that involve sorting or rearranging elements.
- Make sure to take notes on optimizations — you can write a comment listing them all down.

Answer Follow-Ups

- Once you’re done with all of this, you can say — *“and that’s my solution. Do you have any follow up questions or need any final clarifications?”*
- Sometimes, they’ll point out an issue for you to solve or have a more advanced concept they want to test you on.
 - *“how would you handle X input or situation?”*
 - You’ll then have a discussion about that follow-up question using the same principles we discussed earlier, and you might have to modify your code.
- Finally, the interview might end there — or you’ll have a whole extra question to do this process on.

Next Questions

- You very well could have a coding interview with multiple problems, and if that is the case, you should ask initially just so you can budget time accordingly.
- If it's 45 mins for 2 questions, you'll have to speed up each section just so you have time for everything.
- Sometimes you'll have 3-4 easier questions as well — this is just a starting point, you'll have to modify based on your unique circumstances.
- **General Note:** Take Minimal Help
 - You'll want to take as little help as necessary, ideally. For example, something you shouldn't do is ask the interviewer questions like:
 - *"Are there any edge cases I should pay attention to?"*
 - You should have the knowledge to come up with them yourself & ask the interviewer about specifics.
 - *"Are there any tests that fail?"*
 - Don't ask that — YOU need to realize that on your own.

✓ Conclusion

This video, Coding Interview Mastery, is a deep dive on how to handle a classic coding interview from start to finish.

Other videos in this program will teach you about behavioral interviews, asking the interviewer questions, how to prepare for coding interviews, alternative interview styles, etc.

What to take away from this — review this guide several times & make sure you follow it when practicing LeetCode to get used to this style of interviewing.

- You'll also get to practice this through live mock interviews in the Accelerator, which is very helpful.